

Rattrapage de programmation fonctionnelle en Objective Caml

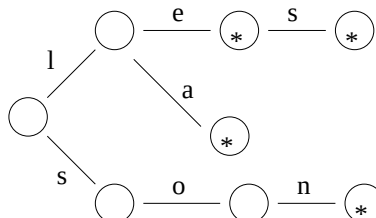
Christian Rinderknecht

19 Juin 2003

Les exercices peuvent être traités de façon indépendante. **Les documents et les calculatrices ne sont pas autorisés.**

1 Lexiques

Un arbre peut représenter une collection de mots de sorte qu'il soit très efficace de vérifier si une séquence donnée de caractères est un mot valide ou pas. Dans ce type d'arbre, appelé un *trie*, les arcs ont une lettre associée, chaque nœud possède une indication si la lettre de l'arc entrant est une fin de mot et la suite des mots partageant le même début. La figure suivante montre le *trie* des mots « le », « les », « la » et « son », l'astérisque marquant la fin d'un mot :



On utilisera le type Caml suivant pour implanter les *tries* :

```
type trie =
  { mot_complet : string option; suite : (char * trie) list }
```

Le type prédéfini `type 'a option = None | Some of 'a` sert à représenter une valeur éventuellement absente. Chaque nœud d'un *trie* contient les informations suivantes :

- Si le nœud marque la fin d'un mot `s` (ce qui correspond aux nœuds étoilés de la figure), alors le champ `mot_complet` contient `Some s`, sinon ce champ contient `None`.

- Le champ `suite` contient une liste qui associe les caractères aux nœuds.
- A.1** Écrire la valeur Caml de type `trie` correspondant à la figure ci-dessus.
- A.2** Écrire une fonction `compte_mots` qui compte le nombre de mots dans un *trie*.
- A.3** Écrire une fonction `select` qui prend en argument une lettre et un *trie*, et renvoie le *trie* correspondant aux mots commençant par cette lettre. Si ce *trie* n'existe pas (parce qu'aucun mot ne commence par cette lettre), la fonction devra lancer une exception `Absent`, que l'on définira. On utilisera la fonction prédéfinie `List.assoc : ∀α, β. α → (α × β) list → β` pour effectuer la recherche dans la liste des sous-arbres *suite*.
- A.4** Écrire une fonction `appartient` qui vérifie si une chaîne de caractères est un mot dans un *trie* donné. La fonction devra prendre un argument supplémentaire `i`, qui représente la position dans le mot associée au nœud courant. Le *i*^e caractère d'une chaîne `s` s'obtient en écrivant `s.[i]`, le premier caractère étant numéroté 0. La longueur de la chaîne `s` s'écrit `String.length s`. On emploiera la fonction `select`.
- A.5** Application : vérificateur orthographique élémentaire. Écrire une fonction `vérifie` qui prend une liste de mots, un lexique et retourne la liste des mots du texte n'apparaissant pas dans le lexique.
- A.6** On désire écrire une nouvelle fonction de recherche qui donne une signification spéciale au caractère point (`.`). Ce caractère se comportera comme un *joker* pouvant remplacer n'importe quelle lettre. Écrire une fonction qui prend en entrée un lexique et un mot pouvant contenir le *joker*, et retourne la liste de tous les mots correspondant.
- A.7** Écrire une fonction qui ajoute un mot à un *trie* (c'est-à-dire qu'elle renvoie un nouveau *trie* contenant en plus ce mot).

2 Tri par fusion (*Merge sort*)

On se propose de programmer un algorithme de tri sur les listes, dit *tri par fusion*. L'idée est de commencer par programmer l'opération de fusion, qui prend deux listes déjà triées et en fait une seule liste triée.

Ensuite, on transforme notre liste à trier, contenant n éléments, en n listes à un élément. Puis on fusionne ces listes deux par deux, ce qui nous donne $n/2$ listes triées à deux éléments, plus éventuellement une liste à un élément (si n est impair). On les fusionne à nouveau deux par deux, etc. jusqu'à ce qu'il ne reste plus qu'une seule liste, qui est alors triée.

Il est intéressant de noter que le type des éléments manipulés n'a pas d'importance, parce que la fonction de comparaison `ordre` sera polymorphe :

elle aura le type $\forall \alpha. \alpha \rightarrow \alpha \rightarrow \text{bool}$. Cela permet d'utiliser la même fonction `mergesort` pour trier des listes d'entiers, de flottants, de listes, etc. sans limitation. De plus, cela permettra de trier par ordre croissant ou décroissant, selon l'implantation de la fonction `ordre`.

- B.1** Écrire une fonction `singletons` qui prend en argument une liste `[x1; ...; xn]` et renvoie la liste des listes à un élément `[[x1]; ...; [xn]]`. On pourra utiliser `List.map :  $\forall \alpha, \beta. (\alpha \rightarrow \beta) \rightarrow \alpha \text{ list} \rightarrow \beta \text{ list}$` qui calcule la liste des images par une fonction donnée d'une liste d'antécédents donnés.
- B.2** Écrire une fonction `merge` qui prend en argument la fonction de comparaison `ordre`, deux listes triées et les fusionne en une seule liste triée.
- B.3** Écrire une fonction `merge2à2` qui prend en argument la fonction de comparaison `ordre`, une liste de listes `[l1; l2; l3; l4; ...]` et renvoie une liste où les listes voisines ont été fusionnées, i.e. `[merge ordre l1 l2; merge ordre l3 l4; ...]`. On prendra garde à traiter correctement le cas où la liste d'entrée est de longueur impaire.
- B.4** En combinant les fonctions précédentes, écrire une fonction nommée `mergesort` qui prend en argument la fonction de comparaison `ordre`, une liste et la trie. Pour cela, on crée la liste des listes à un élément, puis on lui applique `merge2à2` itérativement jusqu'à obtenir une liste de la forme `[l]`. On renvoie alors `l`.
- B.5** Exprimer le nombre maximal de comparaisons nécessité par chacune des fonctions ci-dessus, en fonction de la taille de son argument. En déduire que le tri d'une liste de taille n demande un temps $O(n \cdot \log_2 n)$.

3 Tri rapide (*Quicksort*)

- C.1** Écrire une fonction `partage` qui prend en argument un prédicat `p` et une liste `l` et renvoie un couple formé de la liste des éléments de `l` qui vérifient `p` et de la liste de ceux qui ne le vérifient pas.
- C.2** On choisit un élément `p` de la liste, appelé *pivot*. On sépare ensuite la liste en trois parties : le pivot `p`, la liste `l1` des éléments plus petits que `p`, et la liste `l2` des éléments plus grands que, ou égaux à `p`. (On peut utiliser la fonction `partage` écrite précédemment.) La liste triée est alors égale à la concaténation de `l1` triée, `p` et `l2` triée.